

1. Overview

- Purpose:
- Define functional requirements for building a Delta Live Tables (DLT) pipeline to ingest customer and order CSV data, cleanse and transform it, compute totals, manage an SCD Type 2 ordersummary table, and produce a daily customer aggregate spend table within the gen_ai_poc_databrickscoe catalog and sdlc_wizard schema.

2. Functional Requirements

- FR-DLT-001: Ingest Customer CSV to Delta
- Description: Read customer CSV files from the specified volume path and load them into a managed Delta table.
- Preconditions:
 - Source files available at /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
 - Catalog gen_ai_poc_databrickscoe and schema sdlc_wizard exist or can be created
 - DLT pipeline has service principal/cluster permissions to read volume and write catalog
- Main Flow:
 1. Configure a DLT dataset to read CSVs with header inference for columns CustId, Name, EmailId, Region
 2. Write data to a bronze/silver Delta table named customer in gen_ai_poc_databrickscoe.sdlc_wizard
- Alternate/Exception Flows:
 - If schema inference fails, apply a defined schema with expected data types and continue
 - If source path is empty, produce an empty but valid table version and alert
- Acceptance Criteria:
 - Data from the volume is available as a Delta table gen_ai_poc_databrickscoe.sdlc_wizard.customer with defined schema
 - Read and write operations are logged in DLT event logs
- FR-DLT-002: Ingest Order CSV to Delta
- Description: Read order CSV files from the specified volume path and load them into a managed Delta table.
- Preconditions:
 - Source files available at /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
 - DLT pipeline permissions configured
- Main Flow:
 1. Configure a DLT dataset to read CSVs with header inference for columns OrderId, ItemName, PricePerUnit, Qty, Date, CustId

2. Write data to a bronze/silver Delta table named order in gen_ai_poc_databrickscoe.sdmc_wizard

- Alternate/Exception Flows:
- If malformed records are detected, quarantine to an error dataset and continue
- If source path is empty, create an empty table version and alert
- Acceptance Criteria:
- Data is available as Delta table gen_ai_poc_databrickscoe.sdmc_wizard.order with defined schema
- Malformed records handling is traceable
- FR-DLT-003: Compute TotalAmount in Orders
- Description: Add a computed column TotalAmount = PricePerUnit * Qty to the order table within the DLT pipeline.
- Preconditions:
- FR-DLT-002 completed successfully
- Main Flow:

1. Define a transformation dataset that casts PricePerUnit and Qty to numeric types

2. Compute TotalAmount and persist to the refined orders dataset

- Alternate/Exception Flows:
- If non-numeric values are found, coerce to null and route rows to a data quality expectation for review
- Acceptance Criteria:
- Orders dataset includes a non-nullable TotalAmount where inputs are valid
- Data quality expectations report invalid rows count
- FR-DLT-004: Data Cleansing for Nulls and Duplicates
- Description: Remove records with Null/"Null" textual values in key fields and deduplicate records in both customer and order datasets.
- Preconditions:
- FR-DLT-001 and FR-DLT-002 completed
- Main Flow:

1. Normalize "Null"/"NULL"/null textual representations to actual nulls

2. Filter out records where required fields are null (customer: CustId, Name, EmailId; order: OrderId, CustId, Date)

3. Deduplicate by business keys (customer: CustId; order: OrderId) using latest record by ingestion time

- Alternate/Exception Flows:

- If duplicates have conflicting data, retain the latest by event or ingestion timestamp and log conflicts
- Acceptance Criteria:
- No records with null required keys remain
- No duplicate keys remain in target refined datasets

- FR-DLT-005: Create Ordersummary Table If Not Exists

- Description: Ensure existence of a Delta table ordersummary in the specified catalog and schema.

- Preconditions:

- Catalog gen_ai_poc_databrickscoe and schema sdhc_wizard accessible

- Main Flow:

1. Define a DLT target dataset ordersummary with schema CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date, plus SCD2 metadata columns

2. Configure table properties and expectations as per standards

- Alternate/Exception Flows:

- If table exists with differing schema, apply schema evolution where compatible and log

- Acceptance Criteria:

- ordersummary table exists in gen_ai_poc_databrickscoe.sdhc_wizard with required columns and metadata

- FR-DLT-006: Join Customer and Order Data

- Description: Join cleansed customer and order datasets on CustId to produce enriched order records.

- Preconditions:

- FR-DLT-003 and FR-DLT-004 completed

- Main Flow:

1. Perform inner join of customer and order on CustId

2. Select columns required for ordersummary, including TotalAmount

- Alternate/Exception Flows:

- If a CustId in orders has no customer match, log to a mismatch dataset and exclude from join

- Acceptance Criteria:

- Enriched dataset includes all expected columns and is join-consistent

- FR-DLT-007: Implement SCD Type 2 for Ordersummary

- Description: Manage ordersummary as an SCD Type 2 table, tracking history and changes from the customer dimension.

- Preconditions:

- FR-DLT-005 and FR-DLT-006 available
- Main Flow:
 1. Define SCD2 metadata columns: IsActive (boolean), StartDate (date/timestamp), EndDate (date/timestamp), RecordHash , SurrogateKey
 2. Compare incoming joined records with current active ordersummary records by business keys (CustId, OrderId)
 3. If non-key attribute changes detected for customer attributes (e.g., Name, EmailId, Region), expire existing active records (IsActive=false, set EndDate to current date)
 4. Insert new active records with updated attributes (IsActive=true, StartDate=current date, EndDate=null)
 5. For unchanged records, do nothing
- Alternate/Exception Flows:
 - If multiple active records exist for the same business key, resolve by latest StartDate and flag data issue
 - Acceptance Criteria:
 - Only one active record per CustId+OrderId at a time
 - Historical versions preserved with correct StartDate/EndDate
 - Changes in customer attributes propagate to ordersummary with old rows inactivated and new rows activated
- FR-DLT-008: Compute and Store TotalAmount in Ordersummary
 - Description: Ensure TotalAmount from orders is carried into ordersummary and available for downstream aggregation.
 - Preconditions:
 - FR-DLT-003 and FR-DLT-006 completed
 - Main Flow:
 1. Include TotalAmount in the SCD2 insert/update logic
 2. Validate non-negative TotalAmount values
 - Alternate/Exception Flows:
 - If TotalAmount is null or negative, quarantine the record and exclude from SCD2 upsert
 - Acceptance Criteria:
 - ordersummary contains accurate TotalAmount for active records
 - FR-DLT-009: Create CustomerAggregateSpend Table If Not Exists
 - Description: Create a Delta table customeraggregatespend with columns Name, TotalAmount, Date in the specified catalog and schema.

- Preconditions:
- Catalog and schema accessible
- Main Flow:

1. Define DLT dataset `gen_ai_poc_databrickscoe.sdlc_wizard.customeraggregatespend` with schema Name, TotalAmount, Date

2. Apply basic constraints/expectations on data types and nullability

- Alternate/Exception Flows:
- If table exists with compatible schema, reuse; otherwise evolve schema or fail with alert
- Acceptance Criteria:
- Table exists and is writeable by the pipeline
- FR-DLT-010: Aggregate TotalAmount by Name and Date
- Description: Aggregate TotalAmount from ordersummary grouped by Name and Date and load into customeraggregatespend.
- Preconditions:
- FR-DLT-007 and FR-DLT-009 completed
- Main Flow:

1. Filter ordersummary to active records only

2. Group by Name and Date; sum TotalAmount as TotalAmount

3. Write results to customeraggregatespend using overwrite or merge logic

- Alternate/Exception Flows:
- If no data is available for a given run date, write no rows and log informational event
- Acceptance Criteria:
- Aggregated totals per Name and Date match the sum of active orders in ordersummary
- No duplicate Name+Date rows after load
- FR-DLT-011: Implement Using Delta Live Tables
- Description: Orchestrate all ingestion, transformation, SCD2, and aggregation steps using a single DLT pipeline.
- Preconditions:
- Databricks workspace with DLT enabled and appropriate cluster policy
- Main Flow:

1. Define pipeline settings: target catalog/schema, storage location, continuous/batch mode , development/production mode

2. Implement datasets as DLT tables/views with quality expectations and lineage

3. Configure pipeline schedule/trigger and auto-recovery

4. Deploy and run pipeline; monitor via DLT UI and event logs

- Alternate/Exception Flows:
- On pipeline failure, enable retry with checkpoint restore; send alert to on-call channel
- Acceptance Criteria:
- All datasets are created/updated per run
- DLT event logs show successful execution of each step with data quality metrics
- FR-DLT-012: Data Quality Expectations
- Description: Enforce expectations on schema, nullability, and value ranges across datasets.
- Preconditions:
- DLT pipeline configuration available
- Main Flow:

1. Define expectations for required fields not null (e.g., CustId, OrderId, Date)

2. Validate data types for numeric fields (PricePerUnit, Qty, TotalAmount) and non-negativity

3. Track expectation metrics and drop or quarantine failing rows as configured

- Alternate/Exception Flows:
- If expectation failure rate exceeds threshold , fail pipeline run
- Acceptance Criteria:
- Expectations are visible in DLT UI with pass/fail counts
- Quarantined/error datasets are persisted for diagnostics
- FR-DLT-013: Incremental Processing and Idempotency
- Description: Ensure the pipeline can process new/changed files incrementally and produce idempotent outputs.
- Preconditions:
- Source data arrives in partitioned or timestamped files
- Main Flow:

1. Use Auto Loader or incremental read semantics in DLT for CSV sources

2. Maintain checkpoints to avoid reprocessing

3. Use deterministic merge conditions for SCD2 and aggregates

- Alternate/Exception Flows:

- If reprocessing is required, allow a backfill mode with controlled overwrite
 - Acceptance Criteria:
 - Re-running the pipeline without new data does not change target tables
 - New files are processed exactly once
-
- FR-DLT-014: Governance, Security, and Access Control
 - Description: Apply appropriate permissions and governance controls for all datasets.
 - Preconditions:
 - Unity Catalog enabled and identities provisioned
 - Main Flow:
1. Grant SELECT on consumer tables (ordersummary, customeraggregatespend) to roles
 2. Restrict write access to the DLT service principal
 3. Enable table lineage and audit via Unity Catalog
-
- Alternate/Exception Flows:
 - If permission grant fails, log and notify data governance team
 - Acceptance Criteria:
 - Access controls verified; lineage visible; audit logs populated
-
- FR-DLT-015: Monitoring, Alerting, and Logging
 - Description: Provide operational visibility into the pipeline runs and data health.
 - Preconditions:
 - Workspace has configured notification integrations
 - Main Flow:
1. Capture DLT event logs and expectations metrics
 2. Configure alerts for failures, schema changes, and high data quality failure rates
 3. Expose run status dashboards
-
- Alternate/Exception Flows:
 - If alert delivery fails, retry and log to a fallback sink
 - Acceptance Criteria:
 - Operators receive alerts within minutes of failure
 - Logs enable root-cause analysis for the last days
-
- FR-DLT-016: Metadata and Documentation
 - Description: Document datasets, columns, and business rules for discoverability.

- Preconditions:
- Unity Catalog comments and tags available
- Main Flow:

1. Populate table and column comments for all datasets
2. Tag PII fields (e.g., EmailId) with governance tags
3. Publish a data dictionary in workspace docs

- Alternate/Exception Flows:
- If tagging is restricted, escalate to data governance for approval
- Acceptance Criteria:
- Consumers can discover tables and understand schema and semantics via catalog docs
- FR-DLT-017: Testing and Validation
- Description: Define test cases to validate end-to-end logic, SCD2 behavior, and aggregations.
- Preconditions:
- Test datasets with known outcomes prepared
- Main Flow:

1. Unit-test transformations (TotalAmount, null handling, deduplication)
2. Validate SCD2 updates when customer attributes change for a given CustId
3. Verify aggregate outputs equal expected sums for a test Date

- Alternate/Exception Flows:
- If tests fail, block promotion to production and create defect tickets
- Acceptance Criteria:
- All critical tests pass with evidence stored in CI logs

- FR-DLT-018: Scheduling and SLA
- Description: Define run frequency and data availability SLAs.
- Preconditions:
- Business SLA requirements provided
- Main Flow:

1. Configure pipeline in continuous mode or scheduled batch (e.g., hourly/daily)
2. Set data freshness targets for ordersummary and customeraggregatespend

- Alternate/Exception Flows:

- On SLA breach, trigger incident workflow
 - Acceptance Criteria:
 - Pipeline meets agreed freshness and completeness SLAs in $\geq$ % of runs
-
- FR-DLT-019: Error Handling and Reprocessing
 - Description: Provide mechanisms to handle bad data and support reprocessing.
 - Preconditions:
 - Storage for error records configured
 - Main Flow:
1. Persist quarantined/error rows with error reasons
 2. Provide a parameterized reprocess job to replay errored files/rows
- Alternate/Exception Flows:
 - If reprocess fails repeatedly, escalate with diagnostics
 - Acceptance Criteria:
 - Operators can reprocess impacted data without manual table surgery
-
- FR-DLT-020: Performance and Scalability
 - Description: Ensure the pipeline scales with data volume while maintaining acceptable performance.
 - Preconditions:
 - Expected data volume and growth rate estimates
 - Main Flow:
1. Optimize joins and merges with Z-ordering/statistics
 2. Partition ordersummary and aggregates by Date
 3. Configure autoscaling and proper cluster sizes for DLT
- Alternate/Exception Flows:
 - If runtime exceeds threshold , apply tuning and alert
 - Acceptance Criteria:
 - Pipeline completes within target runtime under peak loads
-
- FR-DLT-021: Backfill and Historical Loads
 - Description: Support initial and ad-hoc historical loads for both source datasets and downstream tables.
 - Preconditions:
 - Historical files available at source paths

- Main Flow:

1. Provide a backfill mode parameter to process historical partitions
2. Ensure SCD2 logic handles back-dated changes with correct StartDate/EndDate semantics

- Alternate/Exception Flows:

- If event-time is missing, use ingestion-time with documented limitations

- Acceptance Criteria:

- Historical loads produce consistent SCD2 history and aggregate results

- FR-DLT-022: Schema Management and Evolution

- Description: Manage schema enforcement and compatible evolution over time.

- Preconditions:

- Schema registry or definitions maintained

- Main Flow:

1. Enforce expected schemas for customer and order on ingest

2. Allow additive evolution with governance approval

3. Version schema changes and communicate to consumers

- Alternate/Exception Flows:

- On incompatible change, fail the run and notify stakeholders

- Acceptance Criteria:

- No silent breaking schema changes occur; schema versions are traceable